Number Guessing Game in Python

A Fun Logic-Based Python Project

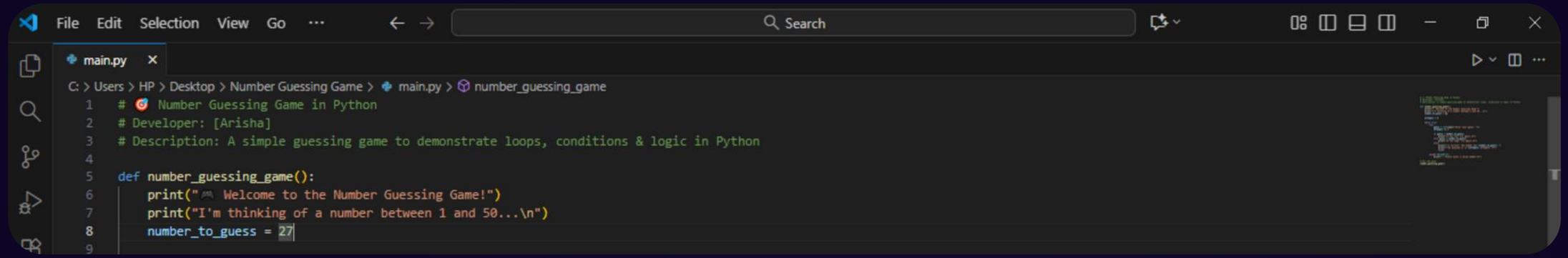
The Engine: Full Python Source Code

The entire game logic is encapsulated within a single function. The secret number is hardcoded as 27 for this demonstration.

main.py

C: > Users > HP > Desktop > Number Guessing Game > main.py > number_guessing_game

```python
# 🎯 Number Guessing Game in Python
# Developer: [Arisha]
# Description: A simple guessing game to demonstrate loops, conditions & logic in Python

def number_guessing_game():
    print("🎮 Welcome to the Number Guessing Game!")
    print("I'm thinking of a number between 1 and 50...\n")
    number_to_guess = 27

    attempts = 0

    while True:
        try:
            guess = int(input("Enter your guess: "))
            attempts += 1

            if guess < number_to_guess:
                print("📉 Too low! Try again.\n")
            elif guess > number_to_guess:
                print("📈 Too high! Try again.\n")
            else:
                print(f"🎉 Correct! The number was {number_to_guess}.")
                print(f"You guessed it in {attempts} attempts! 🏆")
                break

        except ValueError:
            print("⚠️ Please enter a valid number!\n")

# Run the game
number_guessing_game()
```



Live Demo: Step 1 - Initialising the Game

The game starts by greeting the user and setting the context (number between 1 and 50). The variable attempts starts at 0.

1

Game Introduction

The print() statements set the scene for the user.

2

The Secret Number

The number is set to 27. The while True loop begins.

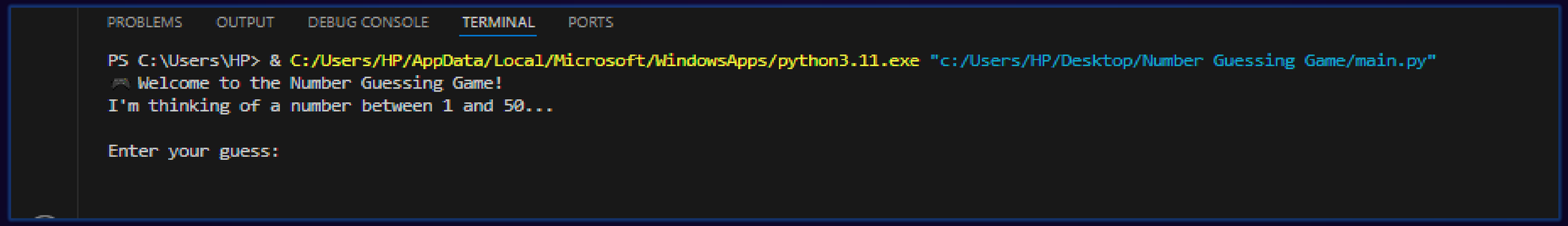
3

Prompt for Input

The program waits for the user's first interaction.

🎮 Welcome to the Number Guessing Game!

I'm thinking of a number between 1 and 50...



Demo in Action: Guess 1 (Too Low)

User Input: 10

The user enters 10. The attempts count is now 1. Since 10 < 27, the first if condition is met.

```python
attempts = 0

while True:
    try:
        guess = int(input("Enter your guess: "))
        attempts += 1

        if guess < number_to_guess:
            print("📉 Too low! Try again.\n")
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Python

PS C:\Users\HP\Desktop\Number Guessing Game> & C:/Users/HP/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/HP/Desktop/Number Guessing Game/main.py"
🎮 Welcome to the Number Guessing Game!
I'm thinking of a number between 1 and 50...

Enter your guess: 10
📉 Too low! Try again.

Enter your guess:

Ln 14, Col 53   Spaces: 4   UTF-8   CRLF   {} Python   3.11.9 (Microsoft Store)
```

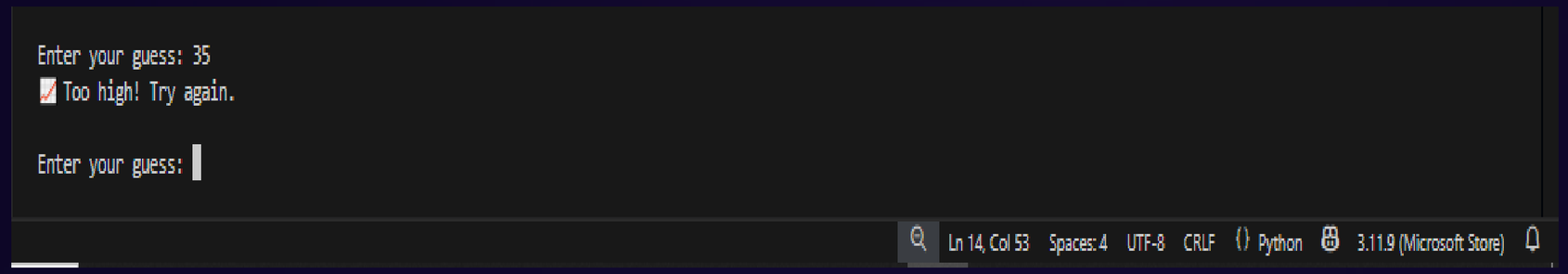
Demo in Action: Guess 2 (Too High)

```
elif guess > number_to_guess:
    print("📈 Too high! Try again.\n")
```

The user tries a much higher number, 35. The attempts count is now 2. Since 35 > 27, the elif condition is met, giving feedback to narrow the search range.

```
Enter your guess: 35
```

📈 Too high! Try again.



Handling Errors: Input Validation

What happens if a user types text instead of a number? The try/except ValueError block catches the error, preventing the program from crashing and providing a helpful message.

The try Block

This section assumes the input will be a valid integer and attempts to convert it using int().

The except Block

If int() fails (e.g., input is "hello"), the ValueError is caught, and the user receives a warning.

```
Enter your guess: twenty
```

Please enter a valid number!

Demo in Action: Guess 3 (The Victory)

🎉 Mission Accomplished! 🏆

Finally, the user enters the correct number, 27. The final else block is executed, printing the success message and exiting the while loop via the break statement.

```
Enter your guess: 27

🎉 Correct! The number was 27.

You guessed it in 3 attempts! 🏆
```

```
        else:
            print(f"🎉 Correct! The number was {number_to_guess}.")
            print(f"You guessed it in {attempts} attempts! 🏆")
            break

    except ValueError:
        print("⚠️ Please enter a valid number!\n")
```

```
Enter your guess: 27
🎉 Correct! The number was 27.
You guessed it in 4 attempts! 🏆
PS C:\Users\HP\Desktop\Number Guessing Game>
```

Takeaways: What We Learnt

The Number Guessing Game beautifully illustrates several foundational Python concepts essential for building interactive applications.

Control Flow

Mastering how the program executes logic based on if/elif/else conditions.

Iteration (Loops)

Using the while True loop to repeat an action until a specific condition is met.

User Interaction

Effectively capturing, converting, and processing user input via input().

Error Robustness

Implementing try/except blocks to handle unexpected input gracefully.

Thank You!

A great first step into Python's logical programming world.